1. Abstract

This project is a web-based simulator for the Halting Problem, one of the best-known undecidable problems in the Theory of Computation. The user can run small programs and Turing machines under a step limit and see one of three outcomes: the program halts, it provably loops forever, or the limit is reached and the answer is unknown. A second module explains the diagonalization argument that proves no general halting decider can exist. A third module accepts a Python while loop written by the user and analyses it.

The application is built with Next.js (App Router), React and TypeScript, and runs entirely in the browser. The report also gives the formal theory behind the tool, transition diagrams and traces for four Turing machines, the architecture of the implementation, and 54 test cases.

2. Introduction, Problem Statement and Objectives

A programmer often asks whether a program will ever finish. In 1936 Alan Turing showed that no single algorithm can answer this question for every program and input. The result is abstract, and students often find it hard to picture. This project makes it concrete by letting the user run programs, watch what happens, and see exactly where a bounded run stops being able to answer.

2.1 Problem statement

Build a tool that runs a given machine or program on an input with a step limit, reports whether it halts or appears to loop forever, and demonstrates why no general algorithm can decide halting for every machine and input pair.

2.2 Objectives

- Show the three outcomes of a bounded run: halts, provably loops, and unknown.
- Explain the diagonalization proof without heavy notation.
- Let the user supply their own Python while loop and get a verdict.
- Handle bad or unexpected input with clear messages, so the tool does not fail during a demo.
- Build a clean, typed and maintainable Next.js code base.

3. Theory of Computation Background

3.1 Turing machine (formal definition)

A deterministic Turing machine (TM) with a single halting state is a 6-tuple M = (Q, Σ, Γ, δ, q0, q_halt), where:

- **Q** is a finite set of states.
- **Σ** is the input alphabet (it does not contain the blank symbol ␣).
- **Γ** is the tape alphabet, with ␣ ∈ Γ and Σ ⊆ Γ.
- **δ** : Q × Γ → Q × Γ × {L, R, S} is the transition function. For example δ(q0, 1) = (q0, ␣, R) means: in state q0 reading 1, write a blank, move right, stay in q0.
- **q0** ∈ Q is the start state.
- **q_halt** ∈ Q is the halting state. The machine stops when it enters q_halt (or when δ is undefined).

In the simulator the blank is written as an underscore (_) and S means “stay”. The tape is unbounded in both directions.

3.2 Configurations and computation

A configuration (instantaneous description) records the current state, the tape contents and the head position. It is written as u q a v, meaning the tape holds u a v, the head is on the symbol a, and the machine is in state q. One computation step moves the machine from one configuration to the next according to δ. A computation is a sequence of configurations C0 ⊢ C1 ⊢ C2 ⊢ … It is halting if some Ck is in state q_halt, and non-halting if it never reaches q_halt.

Since δ is a function, each configuration has exactly one successor. This determinism is the key to the loop detector: if a configuration ever repeats, the whole computation repeats, so the machine runs forever.

3.3 Decidable, recognizable and unrecognizable languages

- **Decidable.** A language L is decidable if some TM halts on every input and accepts exactly the strings in L.
- **Recognizable.** A language L is Turing-recognizable (RE) if some TM accepts exactly the strings in L, but may loop on strings outside L.

Every decidable language is recognizable, and a language is decidable exactly when both it and its complement are recognizable. Figure 1 shows how the classes nest, and where the halting language sits.

3.4 The halting problem

HALT_TM = { ⟨M, w⟩ | M is a Turing machine and M halts on input w }.

The halting problem asks whether HALT_TM is decidable. The answer is no. HALT_TM is recognizable (simulate M on w and accept if the simulation halts) but not decidable. By the rule in 3.3, its complement is therefore not even recognizable.

This is exactly what the simulator shows. Running a program for a while can confirm “halts”, but if it keeps running there is no general way to tell whether it will stop later.

3.5 Proof by diagonalization

Suppose a decider H exists such that H(⟨M, w⟩) = “halts” if M halts on w, and “loops” otherwise, and H itself always halts.

1. Programs can be listed as P1, P2, P3, … and each program can be given as input to another. Figure 2 shows the table of H’s answers: row i, column j is H(Pi, ⟨Pj⟩).
2. Build a new program D that takes a program P as input and runs H(P, ⟨P⟩). If H says “halts”, D loops forever. If H says “loops”, D halts immediately (Figure 3).
3. D is a valid program, so it appears somewhere in the list, say as Pk. Look at what D does on its own description ⟨D⟩.
4. If H(D, ⟨D⟩) says “halts”, D loops forever, so H was wrong. If it says “loops”, D halts, so H was wrong again.
5. Both cases contradict the assumption, so no such H exists. HALT_TM is undecidable.

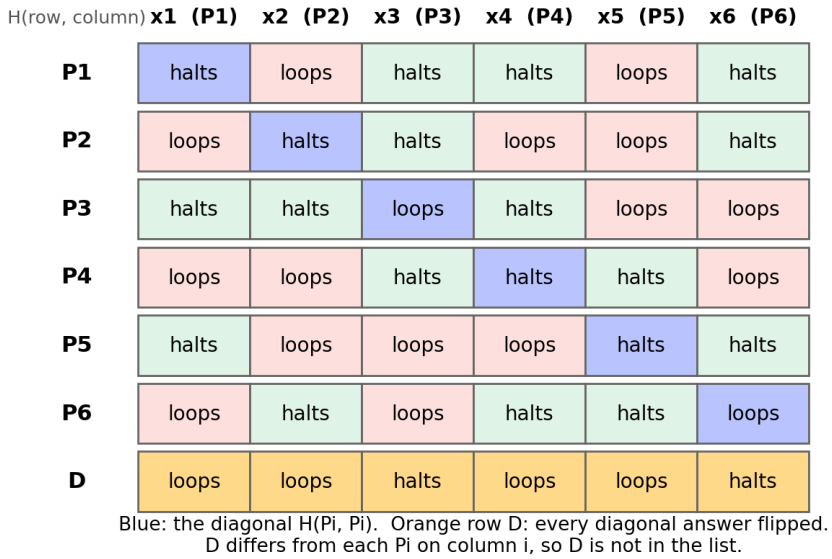


Figure 2: Diagonalization table. Row D flips the diagonal, so it differs from every Pi

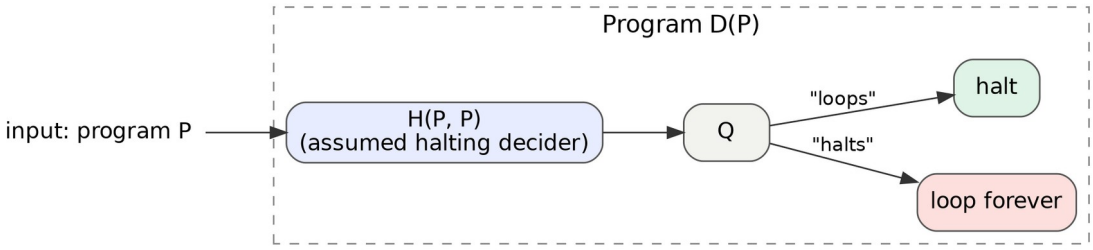


Figure 3: The program D built from the assumed decider H

Figure 4: Reduction A_TM ≤m HALT_TM

3.7 When halting is decidable

The undecidability depends on unbounded memory. For a machine with a finite number of configurations, such as a DFA or a linear bounded automaton (LBA), halting is decidable. Run the machine for more steps than it has configurations; by the pigeonhole principle, if it has not halted by then, some configuration has repeated and it loops forever. The simulator uses the same idea: it records every state it has seen, and a repeat proves an infinite loop. The remaining case is a machine that keeps reaching new configurations, which is where the answer “unknown” comes from.

3.8 Related results

- **Rice’s theorem.** Every non-trivial property of the language a Turing machine recognizes is undecidable. For example, “does this program accept at least one string?” cannot be decided in general.
- **Busy beaver.** The busy beaver function BB(n) is the largest number of 1s (or steps) a halting n-state machine can produce. It grows faster than any computable function, and its values are not computable. Machine M4 in Section 4 is the 3-state champion with 6 ones and 14 steps.
- **Collatz conjecture.** Repeat: if x is even, halve it; if odd, replace it by 3x + 1. Every number tested reaches 1, but no one has proved it for all numbers. For this loop nobody knows whether it halts for every input, so it is a good example of a program the simulator can only answer case by case.
- **Church–Turing thesis.** Python loops with unbounded integers can compute the same functions as Turing machines (two-counter machines are already Turing-complete), so the undecidability result applies to them as well.

4. System Design

4.1 Modules

- **Simulator (tab 1).** Five preset programs: Countdown, Count up forever, Flip-flop, Slow countdown and Collatz. The user sets the start value and the step limit.
- **Story (tab 2).** A step-by-step explanation of the “magic Checker” and the Contrarian program, which is the diagonalization argument in plain words.
- **Loop Checker (tab 3).** The user pastes a Python while loop and gets a verdict, a list of loop values, and a quick static check of the loop condition.

4.2 Architecture

The project uses the Next.js App Router with TypeScript. There is no backend. All three modules run in the browser, which keeps the demo fast and removes any server-side risk from running user code. All three panels stay mounted, so code typed in tab 3 is not lost when the user switches tabs.

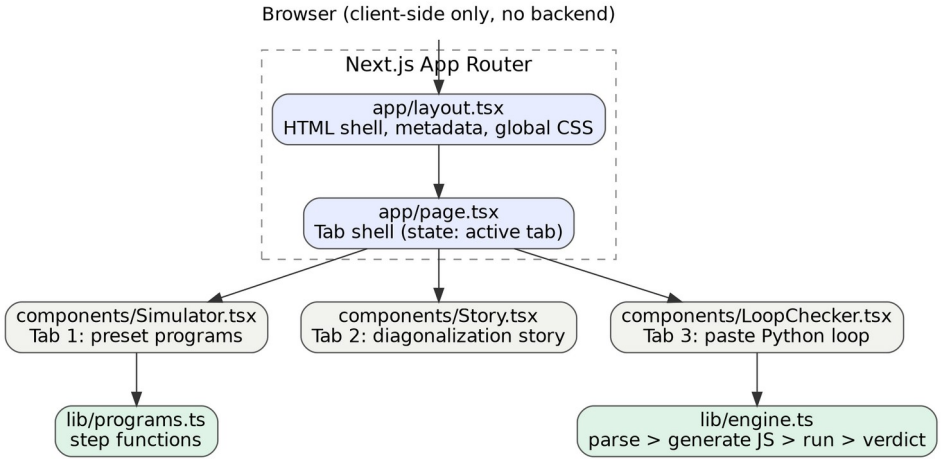


Figure 9: Architecture of the Next.js application

4.3 Technology stack

Layer	Choice
Framework	Next.js 15 (App Router)
UI	React 19 with TypeScript (strict mode)
Styling	Plain CSS with light and dark themes
Backend	None; all logic runs in the browser
Logic engine	Custom Python-subset compiler written in JavaScript (lib/engine.ts)

4.4 Project structure

```
app/
  layout.tsx          HTML shell, metadata, global CSS
  page.tsx            tab shell
  globals.css         theme and layout styles
components/
  Simulator.tsx       tab 1
  Story.tsx           tab 2
  LoopChecker.tsx     tab 3
lib/
  programs.ts         step functions for tab 1
  engine.ts           Python-subset compiler and runner
```

5. Implementation

5.1 Bounded simulation algorithm

Every program in tab 1 is a step function on a state x. It returns the next state, or null when the program halts. Figure 10 shows the algorithm.

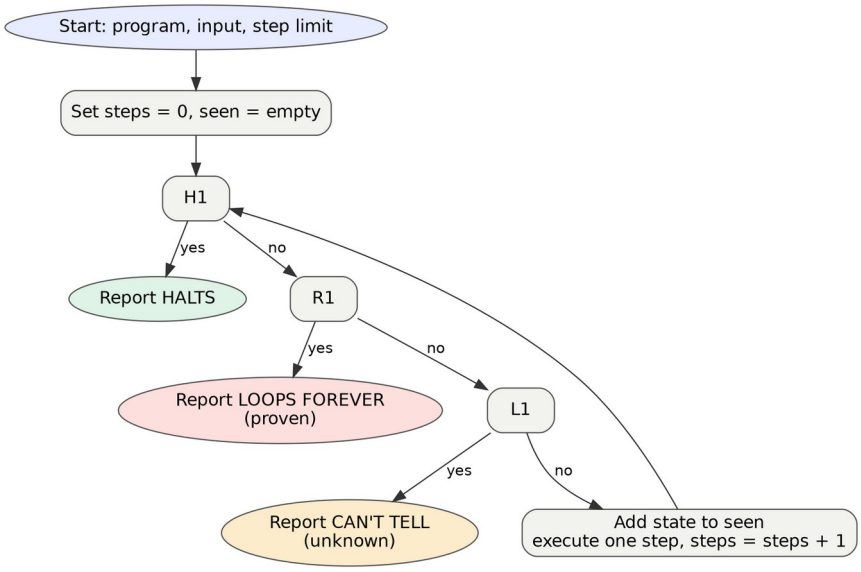


Figure 10: Flowchart of the bounded simulation

```
seen = {}; steps = 0
loop:
    if halted(x):        return HALTS(steps)
    if x in seen:        return LOOPS(seen[x], steps)
    if steps >= limit:   return UNKNOWN
    seen[x] = steps
    x = step(x); steps = steps + 1
```

Correctness. A “halts” verdict is correct because the program really ended. A “loops” verdict is correct because the state x is the whole state of the program and the step function is deterministic, so a repeated x means a repeated future. “Unknown” is the only incomplete outcome. It is the part of the problem that no algorithm can remove.

5.2 Python loop analysis (tab 3)

Tab 3 does not call any external Python. A small compiler turns the pasted code into JavaScript, runs it with a step counter, and watches for repeated states (Figure 11).

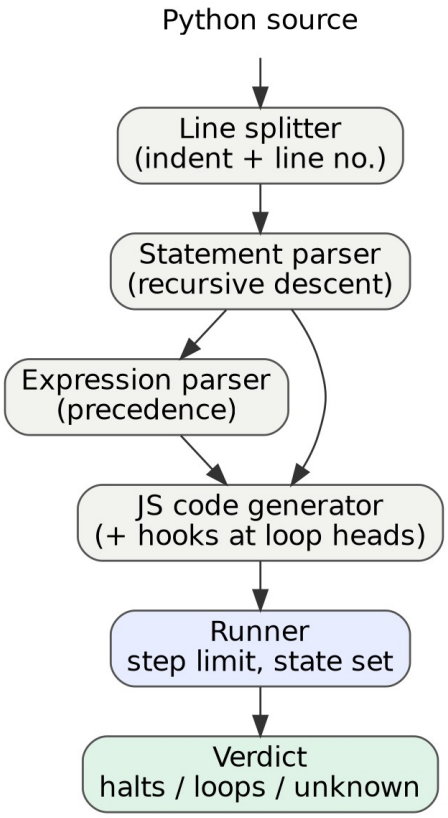


Figure 11: Pipeline of the loop-analysis engine

1. **Line splitter.** Removes comments and blank lines, converts tabs, and keeps each line’s indentation and original line number for error messages.
2. **Statement parser.** A recursive-descent parser reads assignments, tuple assignments, augmented assignments, if/elif/else, while, for-in-range, break, continue, pass and print. It checks that variables are given a value before use.
3. **Expression parser.** A precedence-climbing parser reads expressions and produces JavaScript. Python’s rules are kept: // is floor division, % takes the sign of the divisor, and 0 <= x < 5 is a chained comparison.
4. **Code generator.** Each while loop becomes a JavaScript while(true) with a hook at the top. The hook checks for overflow, records the state, and detects repeats. Each iteration counts as one step.

5. **Runner.** Executes the generated function with a step limit (at most 100,000) and returns the verdict, the iteration count, and the values at each loop check.

State used for loop detection

At each hook the engine records the loop's identifier, the values of all variables, and the counters of any enclosing for loops. If the same combination appears twice, the program is in the same configuration as before and will repeat forever. This matches the TM argument in Section 3.2: the next configuration depends only on the current one.

Safety and input validation

- The pasted text is never run directly. It is parsed into a fixed set of statements, and only whitelisted tokens (numbers, variable names, operators and seven functions) are turned into JavaScript. Quotes, dots, brackets, semicolons and unknown names are rejected with a message.
- Names that would clash with the generated code (for example lim, seen or Math) are rejected as reserved.
- Input is limited to 4,000 characters and 150 lines, and the step limit is clamped to 1 – 100,000.
- Numbers above 10^15 or invalid values (NaN, Infinity) stop the run with "can't tell", since larger values lose integer precision.
- Division or modulo by zero reports a crash, as real Python would raise ZeroDivisionError.
- Every button handler is wrapped in try/catch, so an unexpected failure shows a friendly message instead of a blank screen.

Time and memory. Each loop iteration costs a small constant amount of work plus one hash-map lookup, and memory grows with the number of iterations, which is at most 100,000.

6. Test Cases and Results

Tests are grouped by module. The "Actual result" column for Sections 7.1 to 7.3 was produced by running the same logic as the application, and the Status column compares it with the expected result. Section 7.4 lists interface checks that should be repeated in the browser for screenshots.

6.2 Tab 1: preset programs

#	Program and input	Expected	Actual result	Status
1	Countdown, x = 8, limit 100	Halts after 8 steps	Halts after 8 steps	**PASS**
2	Countdown, x = 0	Halts after 0 steps	Halts after 0 steps	**PASS**
3	Count up forever, x = 0, limit 100	Can't tell (step limit reached)	Can't tell (step limit reached)	**PASS**
4	Flip-flop, x = 0	Loops forever (value 0 at step 0 and step 2)	Loops forever (value 0 at step 0 and step 2)	**PASS**
5	Slow countdown, x = 5000, limit 100	Can't tell (step limit reached)	Can't tell (step limit reached)	**PASS**
6	Slow countdown, x = 5000, limit 5000	Halts after 5000 steps	Halts after 5000 steps	**PASS**
7	Collatz, x = 27, limit 200	Halts after 111 steps	Halts after 111 steps	**PASS**
8	Collatz, x = 97, limit 200	Halts after 118 steps	Halts after 118 steps	**PASS**
9	Collatz, x = 27, limit 50	Can't tell (step limit reached)	Can't tell (step limit reached)	**PASS**
10	Collatz, x = 0 (invalid, treated as 1)	Halts after 0 steps	Halts after 0 steps	**PASS**
11	Countdown, start value left empty (treated as 0)	Halts after 0 steps	Halts after 0 steps	**PASS**
12	Countdown, x = 10^20 (clamped to 10^15), limit 100	Can't tell (step limit reached)	Can't tell (step limit reached)	**PASS**

6.3 Tab 3: pasted Python loops

The first group below contains valid loops, and the second group contains invalid or hostile input that must produce a clear message.

#	Input code	Expected	Actual result	Status

1	`x = 10`
	`while x > 0:`
	`    x = x - 1`
		Halts	Halts after 10 iterations; x=0	**PASS**

2	`a = 48`
	`b = 18`
	`while b != 0:`
	`    a, b = b, a % b`
		Halts	Halts after 3 iterations; a=6, b=0	**PASS**

3	`x = 0`
	`while True:`
	`    x += 1`
	`    if x > 5:`
	`        break`
		Halts	Halts after 6 iterations; x=6	**PASS**

4	`i = 0`
	`while i < 3:`
	`    j = 0`
	`    while j < 3:`
	`        j += 1`
	`    i += 1`
		Halts	Halts after 12 iterations; i=3, j=3	**PASS**

5	`i = 0`
	`while i < 3:`
	`    j = 0`
	`    while j < 3:`
	`        pass`
	`    i += 1`
		Loops forever	Loops forever (proven, loop on line 4)	**PASS**

6	`c = 0`
	`for i in range(3):`
	`    while c < 2:`
	`        c += 1`
		Halts	Halts after 5 iterations; c=2, i=2	**PASS**

7	`x = 1.0`
	`while x > 0.001:`
	`    x = x / 2`
		Halts	Halts after 10 iterations; x=0.0009765625	**PASS**

8	`n = 27`
	`while n != 1:`
	`    n = n // 2 if n % 2 == 0 else 3*n + 1`
		Halts	Halts after 111 iterations; n=1	**PASS**

9	`a = 1`
	`b = 2`
	`while a < 10:`
	`    a, b = b, a + b`
		Halts	Halts after 5 iterations; a=13, b=21	**PASS**

#	Input code	Expected	Actual result	Status

10	`x = 5`
	`while x > 0:`
	`    continue`
		Loops forever	Loops forever (proven, loop on line 2)	**PASS**

11	`while True:` `    pass`	Loops forever	Loops forever (proven, loop on line 1)	**PASS**

12	`x = 5`
	`while x > 0:`
	`    y = 1`
		Loops forever	Loops forever (proven, loop on line 2)	**PASS**

13	`x = 5`
	`while x > 0:`
	`    x = x + 1`
		Can't tell (limit)	Can't tell (step limit reached)	**PASS**

14	`x = 1`
	`while x > 0:`
	`    x = x * 3`
		Can't tell (overflow)	Can't tell (number too big or invalid)	**PASS**

15	`x = 10`
	`while x > 0:`
	`    x = x / 0`
		Crash: division by zero	Stops by crashing (division by zero)	**PASS**

16	`x = 0`
	`for i in range(0, 10, 0):`
	`    x += 1`
	`while x < 1:`
	`    x += 1`
		Crash: range step 0	Stops by crashing (range step is 0)	**PASS**

17	`n = 5`
	`while n:`
	`    n -= 1`
	`    print(n)`
		Halts	Halts after 5 iterations; n=0	**PASS**

18	`x = 0`
	`while 0 <= x < 5:`
	`    x += 1`
		Halts	Halts after 5 iterations; x=5	**PASS**

Summary: 6 of 6 Turing machine tests, 12 of 12 preset tests, and 36 of 36 pasted-code tests gave the expected result. As a performance check, 100,000 loop iterations with two variables took about 201 ms when the engine was run in Node.js.

7. Discussion, Limitations and Future Scope

7.1 What the results show

The tests confirm the three outcomes. “Halts” and “loops” are always correct when reported. “Can’t tell” appears in two kinds of case: a program that really runs forever without repeating (Count up forever, M3), and a program that does halt but needs more steps than the limit (Slow countdown, M4 with limit 5). The tool cannot tell these two apart from the outside, and Section 3.5 proves that no tool can.

7.2 Limitations

- Only a subset of Python is supported: no functions, lists, strings, imports or input().
- Numbers above 10^15 stop the run, so the tool is a finite approximation of unbounded integers.
- Loops whose state keeps changing (for example Collatz-like loops) can only be answered case by case.
- The tool is a practical demonstration. A general halting decider cannot exist.

7.3 Future scope

- Add a Turing machine editor so that users can enter their own transition tables, and draw the diagram automatically.
- Add support for lists, strings and functions.
- Add a ranking-function check (a quantity that strictly decreases and is bounded below) to prove termination for more loops.
- Show a step-by-step visual trace of the variables.
- Add automated tests for the engine with Vitest, and run them in CI.

8. Conclusion

The project turns an abstract theorem into a tool that can be run and inspected. It shows that bounded simulation can prove that a program halts, and can sometimes prove that it loops, but must admit “unknown” for the rest. The diagonalization argument and the reduction from A_TM explain why that last case cannot be removed. The Next.js implementation keeps everything in the browser, validates input carefully, and is organised so that new programs and machines are easy to add.

9. References

1. A. M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem,” Proceedings of the London Mathematical Society, 1936.
2. M. Sipser, Introduction to the Theory of Computation, Cengage Learning.
3. J. E. Hopcroft, R. Motwani and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, Pearson.
4. T. Rado, “On Non-Computable Functions,” Bell System Technical Journal, 1962 (busy beaver).
5. J. C. Lagarias, “The 3x + 1 Problem and its Generalizations,” American Mathematical Monthly, 1985.
6. Next.js documentation, https://nextjs.org/docs
7. React documentation, https://react.dev